POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

À tous mes amis du labos,
vous me manquerez...

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES viii

LIST OF FIGURES ix

LIST OF SYMBOLS AND ACRONYMS x

LIST OF APPENDICES xi

CHAPTER 1 INTRODUCTION 1

CHAPTER 2 LITERATURE REVIEW 3
- 2.1 Canonicalization 4
- 2.2 Matrix stuffing 4
- 2.3 Coefficient extraction 4
- 2.4 Computing derivatives 4
- 2.5 Sparse matrix formats 5

CHAPTER 3 A lightweight canonicalization engine 7

CHAPTER 4 BENCHMARK RESULTS 8

CHAPTER 5 Benchmark results 9

CHAPTER 6 CONCLUSION 10
- 6.1 Synthèse des travaux / Summary of Works 10
- 6.2 Limitations de la solution proposée / Limitations 10
- 6.3 Améliorations futures / Future Research 10

REFERENCES 11

APPENDICES 14

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS AND ACRONYMS

AD	Automatic Differentiation
AML	Algebraic Modeling Language
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SOCP	Second-Order Cone Program
SDP	Semidefinite Program

LIST OF APPENDICES

Appendix A Démo 14

CHAPTER 1 INTRODUCTION

Modeling languages for convex optimization such as CVXPY [1] and YALMIP [2] allow users to formulate optimization problems in a natural mathematical syntax. Through a process called *canonicalization*, these high-level problem descriptions are converted to a *standard conic form* used by many general-purpose solvers

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax + s = b \\ & s \in K, \end{array} \tag{1.1}$$

where $x \in \mathbf{R}^n$ and $s \in \mathbf{R}^m$ are the decision variables, and the *problem data* are the matrix $A \in \mathbf{R}^{m \times n}$, the vectors $b \in \mathbf{R}^m$ and $c \in \mathbf{R}^n$, and the non-empty closed convex cone $K \subseteq \mathbf{R}^m$. Almost all of optimization practice can be divided into two groups, modeling and solving. The modeler's role is to [...]. The solver's role is to find an optimal solution given a problem data instance. Modeling languages serve as the communication bridge between these two groups. When improvements in solvers are made, modeling languages must adapt to allow users to benefit from them. Conversely, it is through natural formulations that modelers can express their needs and help motivate solver improvements.

While the time taken to solve an optimization problem is often dominated by the solver, the time taken to *canonicalize* (or *compile*) a problem can also be significant, especially for large problems. In the case of CVXPY, the *compilation time* largely lies in *matrix stuffing*, the final step of canonicalization where symbolic expressions trees are converted to explicit numerical matrices. Other modeling languages take different approaches to form the problem data, we refer to their approaches as *coefficient extraction* and review them in §2.3.

Matrix stuffing. After expanding nonlinear functions into their *graph implementations* [3], all expressions trees are guaranteed to be affine. We review the entire canonicalization process and show a concrete example in §2.1. At this stage, the convex cone $\mathcal{K}$, which is typically a cartesian product of *standard cones*, is already computed and known. The remaining problem data are the *constraint matrix* A and the vectors b and c. The process to compute b is relatively straightforward, for example, one can *evaluate* the affine expression tree at the origin. The process to obtain c is a special case of the process to obtain A. For these reasons, we will often refer to matrix stuffing as the explicit task of forming A. We review matrix stuffing and its current implementations in CVXPY in §2.2.

Contributions. We make the following contributions:

1. We develop `SparseDiffEngine`.
2. We develop `SparseDiffPy`.
3. We benchmark our methods against existing approaches.

All code is open source and available at `https://github.com/SparseDifferentiation/`.

Thesis outline. The rest of this thesis is organized as follows. In §2 we review the literature on modeling languages, canonicalization, and matrix stuffing. In §3 we describe our proposed canonicalization engine. In §4 we present benchmark results comparing our method to existing approaches. Finally, in §6 we summarize our contributions and discuss future work.

CHAPTER 2 LITERATURE REVIEW

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{2.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [4].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [5]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients. William Orchard-Hays [6] provides a comprehensive history of these early mathematical programming systems, tracing their development from 1952 at RAND Corporation through their widespread adoption on mainframe computers in the late 1960s.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [7]. Commercial systems such as `MaGen` [8], `GAMMA` [9], and `DATAFORM` [10] were among the first matrix generators which were used in various domains.

2.1 Canonicalization

2.2 Matrix stuffing

2.3 Coefficient extraction

`JuMP` and `MathOptInterface`. JuMP is an algebraic modeling language embedded in Julia, originally developed in 2012 by Iain Dunning, Joey Huchette, and Miles Lubin at the MIT Operations Research Center [11, 12]. Its abstraction layer, MathOptInterface (MOI) [13], rethinks how a modeling language communicates with solvers, moving away from the traditional approach of reducing every problem to a single standard form.

Function-set pairs

`AMPL`. [14]

`CasADi`.

2.4 Computing derivatives

Finding the rate of change or the *derivative* of a function at a particular point is a fundamental concept in calculus that was discovered by Leibniz and Newton in the late 1600s. Since then, the underlying theory hasn't changed much, but throughout the years, various methods have been developed to compute derivatives accurately and efficiently. The books by Griewank and Walther [15] and Naumann [16] are excellent references.

Numerical differentiation. []

Automatic differentiation. Jax [17] Pytorch [18] Tensorflow [19]

Forward mode AD.

Reverse mode AD.

Symbolic differentiation. Sympy [20]

Sparse differentiation.

Other methods.

2.5 Sparse matrix formats

Matrices that contain many number of zero entries are known as *sparse matrices*. The simplest sparse matrix data structure is the *coordinate* or *triplet* format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of *nonzero entries* in the matrix.

For example, the matrix

$$\begin{bmatrix} 0 & 0 & 3.2 & 0 \\ 4.1 & 1.8 & 0 & 2.2 \\ 0 & 2.8 & 0 & 3.4 \end{bmatrix} \tag{2.2}$$

can be represented in coordinate format as

```
int    rows[] = {0,  1,   1,   1,   2,   2};
int    cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we use zero-based indexing and array declaration notation from the C programming language [21]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices; the *compressed sparse row* or CSR format is commonly used instead.

Compressed sparse row format. The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (2.2) can be represented in CSR format as

```
int  idxptr[] = {0,  1,              4,        6};
int   cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared to the `rows` array. In the CSR format, accessing a row takes constant time and is very *cache-friendly* since the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for *iterative methods* to solve sparse linear systems [22].

Compressed sparse column format. However, one downside of the CSR format is that accessing a column is much more difficult; one would need to look at every row to find the nonzero entries in a column. An alternative format is the *compressed sparse column* or CSC format, which swaps the role of the `idxptr` array to refer to column indices instead. In contrast to iterative methods, the CSC format is the preferred format for *direct methods* to solve sparse linear systems [23].

Other formats. The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [24] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

Resources. Although the work in this thesis does not necessarily revolve around solving sparse linear systems, a lot of techniques for manipulating sparse matrices we use were inspired by the literature. In particular, the book by Davis [23] was used extensively throughout the development of our methods. The book by Duff [25] is another great reference to learn more about the history and developement of sparse matrix algorithms. Another reference is the review article by the "gang of four" (Gill, Murray, Saunders and Wright) from the Systems Optimization Laboratory at Stanford University [26] which provides a high-level overview of various applications of sparse matrix methods in optimization.

CHAPTER 3 A lightweight canonicalization engine

CHAPTER 4 BENCHMARK RESULTS

CHAPTER 5 Benchmark results

CHAPTER 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

REFERENCES

[1] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[2] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[3] M. Grant and S. Boyd, "Graph Implementations for Nonsmooth Convex Programs," in *Recent Advances in Learning and Control*, ser. Lecture Notes in Control and Information Sciences, V. Blondel, S. Boyd, and H. Kimura, Eds. Springer, London, 2008, vol. 371. [Online]. Available: https://doi.org/10.1007/978-1-84800-155-8_7

[4] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[5] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[6] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[7] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[8] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[9] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[10] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[11] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[12] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[13] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[14] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[15] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[16] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[17] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[18] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library.* Red Hook, NY, USA: Curran Associates Inc., 2019.

[19] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: a system for large-scale machine learning," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'16. USA: USENIX Association, 2016, p. 265–283.

[20] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[21] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[22] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[23] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[24] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[25] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. Oxford, England: Oxford University Press, 1986.

[26] P. E. Gill, W. Murray, M. A. Saunders, and M. H. Wright, "Sparse matrix methods in optimization," *SIAM Journal on Scientific and Statistical Computing*, vol. 5, no. 3, pp. 562–589, 1984.

APPENDIX A DÉMO

Texte de l'annexe A.